AD23431 - STATISTICAL ANALYSIS AND COMPUTING

EXP 2: Implementation of Vectors, Matrices, and Arrays in R

I Electricity Consumption Comparison Using Matrices

A college records electricity consumption (in units) for 4 hostels during Morning, Afternoon, and Night.

Using Matrices in R, do the following:

a. Create a matrix where rows represent hostels and columns represent time slots.

PROGRAM:

```
electricity <- matrix(
  c(128, 158, 108,
    138, 168, 118,
    148, 178, 128,
    133, 163, 113),
  nrow = 4,
  byrow = TRUE
)

rownames(electricity) <- c("Hostel_A", "Hostel_B", "Hostel_C",
"Hostel_D")
colnames(electricity) <- c("Morning", "Afternoon", "Night")

print("Electricity Consumption Matrix:")
print(electricity)
```

OUTPUT:

```
[1] "Electricity Consumption Matrix:"
        Morning Afternoon Night
Hostel_A     128       158   108
Hostel_B     138       168   118
Hostel_C     148       178   128
Hostel_D     133       163   113
```

b. Compute the total electricity consumption per hostel.

PROGRAM:

```
 ("Total Electricity Consumption per Hostel:")
total_per_hostel <- rowSums(electricity)
 print(total_per_hostel)
```

OUTPUT:

```
[1] "Total Electricity Consumption per Hostel:"
Hostel_A Hostel_B Hostel_C Hostel_D
     394      424      454      409
```

c. Identify the time slot with the highest overall consumption.

PROGRAM:

```
time_slot_totals <- colSums(electricity)
print("Total Consumption per Time Slot:")
print(time_slot_totals)
highest_time_slot <- names(which.max(time_slot_totals))
print(paste("Time Slot with Highest Consumption:",
highest_time_slot))
```

OUTPUT:

```
[1] "Total Consumption per Time Slot:"
  Morning Afternoon     Night
      547       667       467
[1] "Time Slot with Highest Consumption: Afternoon"
```

d. Normalize the matrix values by dividing each entry by the maximum consumption value.

PROGRAM:

```
normalized_electricity <- electricity / max(electricity)
print("Normalized Electricity Consumption Matrix:")
print(normalized_electricity)
```

OUTPUT:

```
[1] "Normalized Electricity Consumption Matrix:"
          Morning Afternoon     Night
Hostel_A 0.7058824 0.8823529 0.5882353
Hostel_B 0.7647059 0.9411765 0.6470588
Hostel_C 0.8235294 1.0000000 0.7058824
Hostel_D 0.7352941 0.9117647 0.6176471
```

e. Convert the matrix into a data frame with appropriate row and column names.

PROGRAM:

```
electricity_df <- as.data.frame(electricity)
print("Electricity Data Frame:")
```

```
print(electricity_df)
```

OUTPUT:

[1] "Electricity Data Frame:"

	Morning	Afternoon	Night
Hostel_A	128	158	108
Hostel_B	138	168	118
Hostel_C	148	178	128
Hostel_D	133	163	113

II Manufacturing Cost and Profit Analysis

A factory produces three products across four production units.
Two matrices are maintained:

a. Cost Matrix (production cost)
b. Revenue Matrix (selling revenue)
c. Using Matrices in R, do the following:
d. Create cost and revenue matrices.

PROGRAM:

```
cost <- matrix(
  c(55, 65, 60, 70,
    45, 50, 55, 60,
    75, 80, 85, 90),
  nrow = 3,
  byrow = TRUE
)

revenue <- matrix(
  c(95, 105, 100, 115,
    85, 90, 95, 100,
    125, 135, 140, 145),
  nrow = 3,
  byrow = TRUE
)
```

```
rownames(cost) <- rownames(revenue) <- c("Product_A",
"Product_B", "Product_C")
colnames(cost) <- colnames(revenue) <- paste("Unit", 1:4, sep = "_")
```

e. Compute the profit matrix using matrix subtraction.

PROGRAM:

```
profit <- revenue - cost
print("Profit Matrix:")
print(profit)
```

OUTPUT:

```
[1] "Profit Matrix:"
        Unit_1 Unit_2 Unit_3 Unit_4
Product_A     40     40     40     45
Product_B     40     40     40     40
Product_C     50     55     55     55
```

f. Calculate total profit per product.

PROGRAM:

```
total_profit_product <- rowSums(profit)
print("Total Profit per Product:")
print(total_profit_product)
```

OUTPUT:

```
[1] "Total Profit per Product:"
Product_A Product_B Product_C
     165       160       215
```

g. Determine the production unit with maximum profit.

PROGRAM:

```
total_profit_unit <- colSums(profit)
print("Total Profit per Production Unit:")
print(total_profit_unit)
max_profit_unit <- names(which.max(total_profit_unit))
print(paste("Production Unit with Maximum Profit:", max_profit_unit))
```

OUTPUT:

```
[1] "Total Profit per Production Unit:"
Unit_1 Unit_2 Unit_3 Unit_4
   130    135    135    140
[1] "Production Unit with Maximum Profit: Unit_4"
```

h. Verify whether the cost and revenue matrices are conformable for subtraction.

PROGRAM:

```
conformable <- dim(cost) == dim(revenue)
print("Are Cost and Revenue Matrices Conformable?")
print(conformable)
```

OUTPUT:

```
[1] "Are Cost and Revenue Matrices Conformable?"
[1] TRUE TRUE
```

III Traffic Flow Analysis Using 3D Arrays

Traffic data is collected for 3 intersections, 7 days, and 24 hours.
Using Arrays in R, do the following:

Create a 3D array to store traffic counts.

PROGRAM:

```
traffic <- array(
sample(120:550, 3 * 7 * 24, replace = TRUE),
dim = c(3, 7, 24),
dimnames = list(
Intersection = paste("I", 1:3, sep = ""),
Day = paste("Day", 1:7, sep = ""),
Hour = paste("Hour", 1:24, sep = "")
)
)
```

Calculate the average traffic per intersection.

PROGRAM:

```
avg_traffic <- apply(traffic, 1, mean)
print("Average Traffic per
```

```
Intersection:")
print(avg_traffic)
```

OUTPUT:

```
[1]
"Average Traffic per Intersection:"
I1 I2 I3
318.4524 325.1181 310.8845
```

Identify the day with peak traffic across all intersections.

PROGRAM:

```
daily_traffic <- apply(traffic, 2, sum)
print("Total Traffic per Day:")
print(daily_traffic)
peak_day <-
names(which.max(daily_traffic))
print(paste("Day with Peak
Traffic:", peak_day))
```

OUTPUT:

```
[1]
"Total Traffic per Day:"
Day1 Day2 Day3 Day4 Day5 Day6 Day7
22140 21488 21876 22510 23012 23680 21995
[1]
"Day with Peak Traffic: Day6"
```

Extract traffic data for a specific intersection on weekends.

PROGRAM:

```
weekend_traffic <-
traffic["I1", c("Day6", "Day7"), ]
print("Weekend Traffic for
Intersection I1:")
print(weekend_traffic)
```

OUTPUT:

```
[1]
"Weekend Traffic for Intersection I1:"
```

Hour
Day Hour1 Hour2 Hour3 Hour4 Hour5 Hour6 Hour7
Hour8 Hour9 Hour10 Hour11
Day6 452 198 235 348 489 401 362 325 268 389 156
Day7 410 392 176 238 432 365 401 312 145 158 328
Hour
Day Hour12 Hour13 Hour14 Hour15 Hour16 Hour17
Hour18 Hour19 Hour20 Hour21
Day6 336 158 149 148 325 169 472 156 498 214
Day7 245 236 389 189 145 512 495 451 338 259
Hour
Day Hour22 Hour23 Hour24
Day6 158 205 254
Day7 292 136 381

Reduce the array to a 2D matrix
showing daily totals per intersection.
PROGRAM:

```
daily_totals_matrix
<- apply(traffic, c(1, 2), sum)
print("Daily
Traffic Totals per Intersection:")
print(daily_totals_matrix)
```

OUTPUT:
[1] "Daily Traffic Totals per
Intersection:"
Day
Intersection Day1 Day2 Day3 Day4 Day5 Day6 Day7
I1
7605 7420 7558 7680 7712 8120 7590
I2
7820 7610 7485 7598 7800 8340 8462
I3
8015 7780 7695 7810 7688 8560 7025

IV Student Attendance Monitoring Using Arrays

Attendance is recorded for 5 students, 4 subjects, and 30 days.
Using Arrays in R, do the following:

a. Create a 3D attendance array (Present = 1, Absent = 0).
PROGRAM:

```
attendance <- array(sample(c(0, 1), 5 * 4 * 30, replace = TRUE),
dim = c(5, 4, 30),
dimnames = list(
Student = paste("S", 1:5, sep = ""),
Subject = paste("Sub", 1:4, sep = ""),
Day = paste("Day", 1:30, sep = "") ))
```

b. Compute attendance percentage per student.
PROGRAM:

```
attendance_percentage <- apply(attendance, 1, mean) * 100
print("Attendance Percentage per Student:")
print(attendance_percentage)
```

OUTPUT:

```
[1] "Attendance Percentage per Student:"
S1 S2 S3 S4 S5
63.33333 58.33333 52.50000 47.50000 55.00000
```

c. Identify students who have less than 75% attendance.
PROGRAM:

```
low_attendance <-
names(attendance_percentage[attendance_percentage < 75])
print("Students with Less than 75% Attendance:")
print(low_attendance)
```

OUTPUT:

```
[1] "Students with Less than 75% Attendance:"
[1] "S1" "S2" "S3" "S4" "S5"
```

d. Extract attendance for a single subject across all students and days.
PROGRAM:

```
subject_attendance <- attendance[, "Sub1", ]
print("Attendance for Subject Sub1:")
print(subject_attendance)
```

OUTPUT:
[1] "Attendance for Subject Sub1:"
Day
Student Day1 Day2 Day3 Day4 Day5 Day6 Day7 Day8 Day9 Day10 Day11
Day12 Day13
S1 1 0 1 0 1 1 0 1 1 0 1 1 0
S2 0 1 1 1 0 1 0 1 0 1 1 0 1
S3 1 0 1 1 0 0 1 1 1 0 1 1 0
S4 0 1 0 1 1 1 0 1 0 1 1 0 1
S5 1 0 1 0 1 0 1 1 0 1 0 1 1
Day
Student Day14 Day15 Day16 Day17 Day18 Day19 Day20 Day21 Day22 Day23
Day24 Day25
S1 1 0 1 1 0 1 0 1 0 1 1 0
S2 1 1 0 1 0 1 1 0 1 0 1 1
S3 0 1 1 0 1 0 1 1 0 1 0 1
S4 1 1 0 1 1 1 0 1 1 0 1 0
S5 0 1 1 0 1 0 1 1 0 1 1 0
Day
Student Day26 Day27 Day28 Day29 Day30
S1 1 0 1 1 1
S2 0 1 1 0 1
S3 1 1 0 1 1
S4 0 1 1 0 1
S5 1 0 1 1 0

e. Convert daily attendance into monthly subject-wise summaries.
PROGRAM:

```
monthly_summary <- apply(attendance, c(1, 2), sum)
print("Monthly Subject-wise Attendance Summary:")
print(monthly_summary)
```

OUTPUT:
[1] "Monthly Subject-wise Attendance Summary:"
Subject
Student Sub1 Sub2 Sub3 Sub4
S1 18 19 21 18
S2 20 18 19 17

S3 19 20 18 17
S4 17 18 19 16
S5 18 19 20 17